JAVA SDE-2 INTERVIEW PREPARATION SERIES

STUDY STEP 02 OF 18

Time and Space Complexity for Java Interviews

From First Operation Counts to SDE-2 Trade-off Reasoning

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Build complexity intuition from zero, derive bounds from Java code, reason about memory and collections, then defend optimizations at SDE-2 depth.

GROWTH | BIG-O | LOOPS | SPACE | COLLECTIONS | TRADE-OFFS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `Study Step 01 – Java Problem–Solving Foundations`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume is the bridge from Java syntax to algorithmic problem solving. It begins with concrete operation counts, introduces growth and space gently, then develops the qualified cost models, constraints, invariants, amortization, and optimization explanations expected in SDE-2 interviews.

Readiness map

Decision	Evidence
START HERE	A code snippet has loops, recursion, collection calls, copies, or generated output whose cost is unclear; Constraints may eliminate a brute-force solution; Nested work depends on aggregate pointer movement rather than visual nesting; A data-structure choice changes time, space, order, mutation, or guarantees.
PREPARE FIRST	Java Foundations Study Step 01 through loops, methods, arrays, strings, and basic collections; Ability to dry-run short Java methods; No prior complexity notation is required.
MOVE ON WHEN	Derive time, auxiliary space, and output space from Java code; Qualify best, worst, expected, amortized, and output-sensitive claims; Select Java collections using semantics and defensible cost models; Explain an optimization from baseline through invariant, proof, trade-off, and test evidence.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Study Path Position

01 - Java Problem-Solving Foundations	**YOU ARE HERE** **02 - Time and Space Complexity**	03A - Number Systems and Math Foundations

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Time Complexity from Zero

Learning objectives

By the end of this chapter, you can explain what time complexity measures, choose the correct input size, count work in simple Java code, distinguish constant, logarithmic, linear, linearithmic, and quadratic growth, and use Big-O without treating it as elapsed time.

Why complexity comes after Java basics

Before analyzing code, you must be able to read variables, loops, arrays, methods, and common collections. Complexity asks a second question: as the input grows, how does the amount of work or extra storage grow?

Do not start by memorizing a chart. Start with one concrete input and count what the program actually does.

JAVA

```java
static int first(int[] numbers) {
    return numbers[0];
}
```

For an array of length 10 or 10 million, the method performs one indexed read. The work does not grow with the array length, so it is constant: O(1).

JAVA

```java
static boolean contains(int[] numbers, int target) {
    for (int number : numbers) {
        if (number == target) return true;
    }
    return false;
}
```

In the best case the first element matches. In the worst case the target is last or absent and all `n` values are inspected. Worst-case time is O(n).

Step 1: Name the input size

Complexity is meaningless until its dimensions are named.

- For `int[] numbers`, let `n = numbers.length`.
- For a string, `n` may mean UTF-16 code units unless the contract says code points.
- For a matrix, use `rows` and `columns`, or total cells when that is the true work.
- For two independent arrays, use `n` and `m` instead of pretending they are equal.
- For a graph, use `V` vertices and `E` edges.
- For returned matches, use `k` for output size when output affects work or memory.

Example with two independent inputs:

JAVA

```java
static int countEqualPairs(int[] left, int[] right) {
    int count = 0;
    for (int first : left) {
        for (int second : right) {
            if (first == second) count++;
        }
    }
    return count;
}
```

If lengths are `n` and `m`, every pair is compared, so time is O(nm), not automatically O(n squared).

Step 2: Choose a meaningful operation

Count the operation that grows: element visits, comparisons, map lookups, recursive calls, or generated results. Exact statement counts are useful for learning, but Big-O keeps the dominant growth.

JAVA

```java
static int sum(int[] numbers) {
    int total = 0;                    // 1 initialization
    for (int number : numbers) {      // n visits
        total += number;              // n additions
    }
    return total;                     // 1 return
}
```

A simplified count is `2n + 2`. As `n` grows, the linear part dominates, so time is O(n). Big-O does not claim the method literally takes `n` nanoseconds.

Step 3: See the common growth shapes

O(1): constant growth

JAVA

```java
int middle = numbers[numbers.length / 2];
```

One indexed read. The array still occupies O(n) input space, but this expression uses O(1) time and O(1) auxiliary space.

O(log n): repeatedly shrink the remaining problem

JAVA

```java
int value = n;
int steps = 0;
while (value > 1) {
    value /= 2;
    steps++;
}
```

For `n = 16`, values are 16, 8, 4, 2, 1: four divisions. Doubling `n` adds roughly one step. The loop is O(log n). Unless a base matters to the algorithm's meaning, logarithm bases differ only by a constant factor.

O(n): visit each input item a constant number of times

JAVA

```java
for (int number : numbers) {
    process(number);
}
```

This is O(n) only if `process` is O(1). A method call is not automatically constant; include the work inside it.

O(n log n): linear work across logarithmic levels

Comparison sorting is a familiar example:

JAVA

```java
Arrays.sort(values);
```

For ordinary interview analysis, sorting `n` primitive values is O(n log n) worst-case under the documented Java sorting algorithm family, while exact implementation details and auxiliary space depend on the overload/type. State the API and data type rather than repeating one universal sorting claim.

O(n squared): compare many pairs

```java
for (int left = 0; left < numbers.length; left++) {
    for (int right = left + 1; right < numbers.length; right++) {
        compare(numbers[left], numbers[right]);
    }
}
```

The comparisons total `(n - 1) + (n - 2) + ... + 1 = n(n - 1)/2`, which grows quadratically: O(n ^ 2).

O(2^n) and O(n!): enumerate choices or arrangements

Generating every subset can produce `2^n` outputs. Generating every permutation can produce `n!` outputs. If the algorithm must materialize all of them, the output itself already requires that much work. These bounds are not automatically mistakes; they may be unavoidable for the requested output.

Big-O, Big-Omega, and Big-Theta gently

- **O(g(n))** is an asymptotic upper bound: growth is no faster than a constant multiple of `g(n)` after some point.
- **Omega(g(n))** is a lower bound.
- **Theta(g(n))** gives matching upper and lower growth.

In interviews, "the complexity is O(n)" often informally means a tight worst-case bound. Be precise when the distinction matters. A full scan that always reads all `n` elements is Theta(n). A contains search is O(n) worst-case but can return in Theta(1) best-case.

Drop constants and lower-order terms only after deriving

- O(3n + 10) becomes O(n).
- O(n ^ 2 + 20n + 100) becomes O(n ^ 2).
- O(n + m) does not become O(n) when `m` is independent.
- O(n log n + n) becomes O(n log n).

Dropping terms is the last step, not the first. First show where the terms came from so an interviewer can verify the reasoning.

Best, average/expected, and worst cases

Say which case you are reporting.

For linear search:

- best case: target first, O(1);
- worst case: target last/absent, O(n);
- expected case: requires a stated distribution of target positions and presence.

"Average" is not a magic middle answer. Expected analysis needs a probability model. Interview and production capacity decisions usually begin with worst case, then add expected or amortized behavior when relevant.

A first constraint-to-complexity map

These are heuristics, not promises; constants, language, time limit, and operations matter.

Approximate input scale	Often worth considering
`n <= 10`	factorial/backtracking may be possible
`n <= 20–25`	subset O(2^n) may be possible
`n <= 1,000`	O(n^2) may be possible
`n <= 100,000`	O(n log n) or O(n) is typical
`n` in millions	usually O(n) with careful memory/constants

Never quote this table without considering the operation cost, number of test cases, memory, and environment.

Common beginner mistakes

- Counting source lines instead of executed work.
- Calling array access O(n) because the array contains n elements.
- Calling every method O(1) without analyzing its body/API.
- Multiplying loops merely because one appears inside another.
- Replacing two independent sizes with one `n`.
- Reporting the best case without labeling it.
- Saying O(n) means exactly n operations or a fixed duration.
- Ignoring output work.
- Optimizing before a correct baseline exists.

Quick check

1. What is the input size for a method receiving two arrays of different lengths?
2. Why is `numbers[index]` O(1)?
3. How many times can a repeated-halving loop run?
4. Why does `n(n - 1)/2` become O(n squared)?
5. What probability assumption supports an expected-case claim?

Practice

1. **Foundation:** Count exact body executions for loops of length 0, 1, 4, and n.
2. **Foundation:** Label five snippets as O(1), O(log n), O(n), O(n log n), or O(n squared).
3. **Interview Core:** Analyze a method that scans arrays of lengths n and m sequentially.
4. **Interview Core:** Explain why generating all subsets needs at least Omega(2 ^ n) output work.
5. **SDE-2 Follow-up:** Given n = 100,000, compare a quadratic baseline with a sorting-based solution and state what evidence is still missing.

RECAP | Chapter summary

Complexity starts with a named input size and a count of executed work. Constant access, repeated halving, full scans, sorting-level work, and pair enumeration create recognizable growth shapes, but each claim must follow the code and its contracts. Big-O compares growth; it is not a stopwatch.

SDE-2 CORE CHAPTER

Chapter 2 - Reading Time Complexity from Java Code

Learning objectives

This chapter teaches a repeatable method for analyzing Java statements, consecutive phases, branches, nested and dependent loops, helper calls, early exits, sorting, strings, and output-producing code.

The five-step reading method

For every snippet:

1. Name each independent input dimension.
2. Identify the operation whose count grows.
3. Write a count or sum before simplifying.
4. Include called methods and library APIs.
5. State the case and assumptions.

Consecutive phases add

JAVA

```java
static int summarize(int[] values) {
    int negatives = 0;
    for (int value : values) {
        if (value < 0) negatives++;
    }

    int positives = 0;
    for (int value : values) {
        if (value > 0) positives++;
    }
    return negatives + positives;
}
```

The loops run `n` and `n` times: O(n + n) = O(n). They are not nested, so they do not multiply.

If the phases use independent arrays of lengths `n` and `m`, the result is O(n + m), not O(max(n,m)) unless you explicitly use an equivalent bound and preserve the meaning of both inputs.

Branches usually take the maximum reached cost

JAVA

```java
if (alreadySorted(values)) {       // O(n)
    return copy(values);           // O(n)
}
Arrays.sort(values);               // O(n log n)
return values;
```

Worst-case time is O(n log n), because only one branch path executes after the condition and sorting dominates. The sorted branch is O(n), but worst-case reporting takes the most expensive reachable path. Include the condition cost.

Independent nested loops multiply

JAVA

```java
for (int row = 0; row < rows; row++) {
    for (int column = 0; column < columns; column++) {
        visit(row, column);
    }
}
```

The body runs `rows * columns` times: O(rc). For a square `n x n` matrix this becomes O(n squared), but do not assume square shape unless the contract says so.

Triangular loops are still quadratic

JAVA

```java
for (int left = 0; left < n; left++) {
    for (int right = left + 1; right < n; right++) {
        compare(left, right);
    }
}
```

Counts are `(n - 1) + (n - 2) + ... + 1`, approximately `n^2 / 2`. Dropping the constant still gives O(n squared).

Nested-looking loops can be linear

JAVA

```java
int left = 0;
for (int right = 0; right < values.length; right++) {
    while (left <= right && windowTooLarge(left, right)) {
        left++;
    }
}
```

Do not multiply `n` by `n` automatically. `right` moves forward at most n times and `left` also moves forward at most n times across the entire method. Total pointer movements are at most 2n, so the control movement is O(n), assuming `windowTooLarge` is O(1).

The proof uses an aggregate count: each pointer never moves backward.

Dependent inner bounds require a sum

JAVA

```java
for (int size = 1; size <= n; size *= 2) {
    for (int index = 0; index < size; index++) {
        work();
    }
}
```

The outer loop has log n iterations, but the inner work is `1 + 2 + 4 + ... + n`, a geometric series bounded by 2n. Total is O(n), not O(n log n).

Contrast:

JAVA

```java
for (int size = 1; size <= n; size *= 2) {
    for (int index = 0; index < n; index++) {
        work();
    }
}
```

Now each of log n outer iterations performs n work, so total is O(n log n).

Early exits change best case, not necessarily worst case

JAVA

```java
for (int index = 0; index < values.length; index++) {
    if (values[index] == target) return index;
}
```

Best case O(1), worst case O(n). If an interviewer asks for typical behavior, define a distribution instead of saying "average O(n/2), therefore O(n)" without assumptions.

Include helper and API costs

```java
for (String word : words) {
    if (word.contains(pattern)) matches++;
}
```

The loop is not simply O(n). If there are n words, average word length L, and pattern length p, substring-search cost must be included according to the API/algorithm model. A safe interview bound might be O(total text examined times pattern-related cost), with a simpler O(nL p) naive model when explicitly assumed.

Likewise:

```java
for (int value : values) {
    list.contains(value);
}
```

For an ArrayList of size growing to n, each `contains` can be O(n), making the loop O(n squared). Replacing the lookup side with HashSet can make membership expected O(1) and total expected O(n), using O(n) extra space.

Hidden work in convenient syntax

String concatenation

```java
String result = "";
for (String word : words) {
    result += word;
}
```

String is immutable. Repeatedly copying a growing result can make total work quadratic in the final character count. Use `StringBuilder` for repeated construction:

```java
StringBuilder builder = new StringBuilder();
for (String word : words) builder.append(word);
String result = builder.toString();
```

This is linear in the characters appended, excluding domain-specific formatting work.

Copies and conversions

`Arrays.copyOf(array, n)`, `new ArrayList<>(collection)`, `List.copyOf(collection)`, and `stream().toList()` traverse/copy membership. A single method call can still be O(n) and allocate O(n) storage.

Sorting before a scan

JAVA

```java
Arrays.sort(values);       // O(n log n)
for (int value : values) { // O(n)
    process(value);
}
```

Total is O(n log n + n) = O(n log n). Do not discard the scan before showing the sum; it may matter for concrete work or if sorting assumptions change.

Matrix and jagged-array analysis

For a rectangular `r x c` matrix, visiting every cell is O(rc).

For a jagged array, use total cells:

JAVA

```java
int cells = 0;
for (int[] row : matrix) {
    if (row == null) continue;
    for (int value : row) {
        consume(value);
        cells++;
    }
}
```

Time is O(total cells), often written O(sum of row lengths). `rows * maximumColumns` is a valid upper bound but can be loose.

Multiple test cases

If there are `t` test cases each of size `n`, O(tn) may be appropriate. If sizes vary, a tighter expression is O(n1 + n2 + ... + nt). Online judges often constrain the sum of input sizes, which can make this distinction important.

Output-sensitive time

JAVA

```java
static List<int[]> allMatchingPairs(int[] values) {
    List<int[]> result = new ArrayList<>();
    for (int left = 0; left < values.length; left++) {
        for (int right = left + 1; right < values.length; right++) {
            if (matches(values[left], values[right])) {
                result.add(new int[] {left, right});
            }
        }
    }
    return result;
}
```

Search work is O(n squared). Creating and returning k pairs adds O(k) time and output space. When an algorithm otherwise performs O(n) work plus emits k results, report O(n + k).

Dry-run table: four patterns

Code shape	Count before simplification	Result
two consecutive n loops	n + n	O(n)
independent n by m loops	nm	O(nm)
two forward-only pointers	at most n + n moves	O(n)
doubling level, full n scan	n log n	O(n log n)

WATCH OUT | Common mistakes

- Multiplying consecutive loops.
- Failing to multiply genuinely independent nested loops.
- Ignoring a helper/API cost.
- Treating an early return as proof of O(1) worst case.
- Calling a two-pointer loop quadratic without aggregate reasoning.
- Calling every doubling-loop combination O(n log n) without summing dependent work.
- Using rows times first-row length for a jagged matrix.
- Ignoring repeated copies or immutable-string growth.
- Dropping multiple independent input variables.

Practice

1. **Foundation:** Analyze three consecutive loops with lengths n, n, and m.
2. **Foundation:** Derive the exact number of pairs in a triangular nested loop.
3. **Interview Core:** Prove a forward-only sliding window is O(n) by counting pointer moves.
4. **Interview Core:** Compare a list-membership nested pattern with a HashSet alternative.
5. **Interview Core:** Analyze repeated string concatenation by final output length.
6. **SDE-2 Follow-up:** Express the cost of processing variable-sized test cases using a sum rather than one maximum.

RECAP | Chapter summary

Read Java complexity by execution: phases add, independent nesting multiplies, dependent bounds require sums, early exits need case labels, and helper/API work must be included. Aggregate movement, total cells, total text, and output size often provide a more accurate dimension than the visible number of loops.

SDE-2 CORE CHAPTER

Chapter 3 - Space Complexity from Zero

Learning objectives

By the end of this chapter, you can separate input, output, and auxiliary space; identify peak live storage; include recursion stacks and hidden copies; and explain when an in-place algorithm is worth mutating its input.

Start with three different questions

Space analysis becomes confusing when three quantities are mixed together.

1. **Input space:** storage already occupied by the caller's input.
2. **Auxiliary space:** extra storage the algorithm uses while working.
3. **Output space:** storage required for the result that the contract asks the method to produce.

Interviewers usually mean auxiliary space when they ask, "What is the space complexity?" State that convention instead of making them guess.

JAVA

```java
static int maximum(int[] numbers) {
    if (numbers.length == 0) {
        throw new IllegalArgumentException("numbers must not be empty");
    }

    int best = numbers[0];
    for (int number : numbers) {
        best = Math.max(best, number);
    }
    return best;
}
```

For `n = numbers.length`:

- the input array occupies O(n) input space;
- the method adds only `best`, `number`, and loop state, so auxiliary space is O(1);
- the returned integer is O(1) output space.

Calling the method "O(n) space" without a qualifier hides the useful fact that it does not allocate storage proportional to the input.

Constant versus growing auxiliary storage

A fixed number of variables is O(1)

JAVA

```java
static void reverseInPlace(int[] values) {
    int left = 0;
    int right = values.length - 1;

    while (left < right) {
        int temporary = values[left];
        values[left] = values[right];
        values[right] = temporary;
        left++;
        right--;
    }
}
```

The method uses a fixed number of variables regardless of `n`, so auxiliary space is O(1). The time is O(n). It mutates the caller's array, which is part of the method contract-not a detail to hide behind the word "optimal."

A copy grows with the input

JAVA

```java
static int[] reversedCopy(int[] values) {
    int[] result = new int[values.length];
    for (int index = 0; index < values.length; index++) {
        result[values.length - 1 - index] = values[index];
    }
    return result;
}
```

The returned array contains `n` integers. If output is excluded, auxiliary working space is O(1); if all newly allocated result storage is reported, space is O(n). Say which convention you use. In practice, the output allocation still matters to memory capacity.

Peak live storage, not total allocations over history

Space complexity describes the maximum storage simultaneously needed during an execution.

JAVA

```java
static void processInBatches(int[] values, int batchSize) {
    for (int start = 0; start < values.length; start += batchSize) {
        int end = Math.min(values.length, start + batchSize);
        int[] batch = Arrays.copyOfRange(values, start, end);
        consume(batch);
    }
}
```

If each completed `batch` becomes unreachable before the next allocation and `consume` does not retain it, peak additional array storage is O(min(n, `batchSize`)), even though many arrays may be allocated over the full run. Allocation rate can affect garbage collection, but it is different from peak auxiliary space.

If `consume` stores every batch, the retained space can grow to O(n). Ownership and lifetime change the answer.

Arrays, matrices, and collections

- `new int[n]` adds O(n) elements of storage.
- `new boolean[rows][columns]` adds O(rows times columns) elements for a rectangular matrix.
- A jagged `int[][]` should be described by the sum of row lengths, not automatically O(rows times maximumColumns).
- A map or set holding up to `u` distinct keys uses O(u) logical entries; implementation overhead exists, but exact bytes are JVM- and implementation-dependent.
- `new ArrayList<>(existingList)` copies `n` references: O(n) additional logical slots, not `n` deep copies of the referenced objects.

Avoid claiming exact object sizes unless the environment, JVM options, object layout, and measurement method are specified. Asymptotic interview analysis normally counts logical elements or frames.

Aliasing can avoid a copy-but changes the contract

JAVA

```java
static int[] identity(int[] values) {
    return values;
}
```

The returned reference points to the same array; there is no element copy. Auxiliary space is O(1). But callers can now observe shared mutation.

JAVA

```java
static int[] defensiveCopy(int[] values) {
    return Arrays.copyOf(values, values.length);
}
```

This takes O(n) time and O(n) result storage, but protects the original array from mutation through the returned reference. Space is not merely a score to minimize; it can buy ownership safety.

Recursion uses call-stack space

Each unfinished recursive call needs a frame holding return state and local information. Exact bytes are JVM-dependent, but the number of simultaneously active calls gives an asymptotic bound.

Linear recursion depth

JAVA

```java
static long factorial(int n) {
    if (n < 0) throw new IllegalArgumentException("n must be non-negative");
    if (n <= 1) return 1L;
    return n * factorial(n - 1);
}
```

For `n = 4`, the active chain reaches `factorial(4)`, `factorial(3)`, `factorial(2)`, `factorial(1)`. Depth is O(n), so auxiliary stack space is O(n). Java does not guarantee tail-call optimization, so a tail-recursive rewrite does not justify O(1) stack space.

Logarithmic recursion depth

JAVA

```java
static int binarySearch(int[] sorted, int target, int low, int high) {
    if (low > high) return -1;
    int middle = low + (high - low) / 2;
    if (sorted[middle] == target) return middle;
    if (sorted[middle] < target) {
        return binarySearch(sorted, target, middle + 1, high);
    }
    return binarySearch(sorted, target, low, middle - 1);
}
```

The remaining range halves each time. Time is O(log n) and recursive stack space is O(log n). An iterative binary search can preserve O(log n) time with O(1) auxiliary space.

Branching does not automatically mean exponential stack space

JAVA

```java
static int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

This implementation makes exponentially many calls over time, but the deepest active chain is only O(n). Therefore time is O(2^n) as a simple upper bound, while stack space is O(n). Space counts simultaneous frames, not every call ever created.

Trees use height, not always node count

A depth-first traversal of a tree has O(h) call-stack space, where `h` is height. A balanced tree has `h = O(log n)`; a skewed tree can have `h = O(n)`. Report O(h) first, then discuss shapes.

Output-sensitive space

JAVA

```java
static List<Integer> positionsOf(int[] values, int target) {
    List<Integer> positions = new ArrayList<>();
    for (int index = 0; index < values.length; index++) {
        if (values[index] == target) positions.add(index);
    }
    return positions;
}
```

Let `k` be the number of matches. Time is O(n); result space is O(k). The list's capacity policy and boxed `Integer` objects add concrete overhead, but O(k) is the useful asymptotic model. If the contract requires all positions, Omega(k) output space is unavoidable.

Hidden space in Java code

Boxing

`List<Integer>` stores references to wrapper objects, not primitive `int` elements. This is still O(n) logical space, but can be materially larger than `int[]`. Mention the representation when memory limits are tight.

Substrings and conversions

Do not assume `substring`, `toCharArray`, `split`, streams, or collectors are zero-copy views. Model them according to the selected Java version and documented API behavior. In modern Java, a new substring has its own character storage rather than retaining a view of the entire original backing array.

Sorting

Do not give one universal auxiliary-space claim for `Arrays.sort`. Primitive-array and object-array overloads use different algorithm families. State the overload/type or keep the answer qualified.

Views versus copies

`list.subList(...)` is a backed view; `new ArrayList<>(list.subList(...))` creates a new list. A view can use little new storage while retaining and sharing a larger structure. Low allocation does not imply independent ownership.

A repeatable space-analysis method

1. Name input dimensions.
2. Decide whether the question asks for auxiliary, output, or total storage.
3. List allocations whose size grows with input.
4. Count maximum simultaneously active recursion frames.
5. Include hidden library copies, boxing, and retained views where relevant.
6. Express space with the most accurate dimension: O(n), O(rows times columns), O(h), O(k), or a sum.
7. State mutation and ownership trade-offs.

TRACE IT | Dry run: breadth-first traversal frontier

Suppose a queue-based tree traversal processes one level at a time.

Moment	Queue contents	Live queued nodes
start	root	1
after root	its children	2
middle level	next frontier	up to width
finish	empty	0

Time is O(n) because each node enters and leaves once. Auxiliary space is O(w), where `w` is maximum tree width-not automatically O(n), although O(n) is a valid worst-case upper bound.

WATCH OUT | Common mistakes

- Saying every iterative method is O(1) space.
- Ignoring recursion frames.
- Counting every historical allocation instead of peak live storage.
- Treating returned output as free without stating the convention.
- Calling shallow copies deep copies.
- Assuming a reference copy duplicates an object.
- Claiming Java optimizes tail recursion.
- Reporting a tree traversal as O(log n) space without a balance guarantee.
- Calling mutation "better" without discussing caller expectations.

Quick check

1. What is the difference between input space and auxiliary space?
2. Why does naive recursive Fibonacci use O(n), not O(2^n), stack space?
3. What space dimension describes a tree DFS most accurately?
4. When can a batch-processing loop use less peak space than total allocated bytes?
5. Does copying a list of object references copy the objects?
6. Why might O(n) boxed storage matter more than O(n) primitive storage?

Practice

1. **Foundation:** Analyze the auxiliary and output space of an array-filter method.
2. **Foundation:** Compare `return values` with `return Arrays.copyOf(values, values.length)`.
3. **Interview Core:** Analyze recursive and iterative binary search space.
4. **Interview Core:** Give time and space bounds for a frequency map with `u` distinct keys.
5. **Interview Core:** Analyze a jagged matrix using total cell count.
6. **SDE-2 Follow-up:** Choose between an in-place sort and a defensive copy when an API promises not to mutate caller data.

RECAP | Chapter summary

Report space with a contract: input, auxiliary, and output are different. Count peak live storage, growing allocations, and active recursion depth. Then explain what memory buys-ownership, speed, simpler code, or required output-instead of treating O(1) space as automatically superior.

SDE-2 CORE CHAPTER

Chapter 4 - Java Collections Cost Models for Interviews

Learning objectives

This chapter connects the collection APIs learned in Java Fundamentals to interview complexity. You will choose a concrete implementation, qualify worst-case versus expected or amortized costs, include iteration and conversion work, and avoid common "all map operations are O(1)" mistakes.

Start with behavior, then cost

Do not choose a collection from a complexity table alone. Ask:

1. Do duplicates matter?
2. Must insertion order or sorted order be preserved?
3. Do you need lookup by numeric index, by key, or by priority?
4. Where do insertions and removals occur?
5. Is a worst-case guarantee required, or is expected performance acceptable?
6. What equality, ordering, and mutation rules apply to elements or keys?

Only then compare costs.

Cost vocabulary

- **Worst-case:** an upper bound for any valid input/state under the stated model.
- **Expected:** an average over a stated source of randomness or distribution. Hash-based collections are commonly described with expected O(1) lookup under a sound hash distribution, not a universal guarantee.
- **Amortized:** average cost per operation across a sequence, even if an occasional individual operation is expensive. Array growth is the standard example.
- **Output-sensitive:** includes the number k of returned or visited results.

ArrayList: contiguous indexed sequence

Use `ArrayList` as the default general-purpose `List` when you need indexed access and append-heavy use.

Operation	Typical asymptotic cost	Reason
`get(i)`, `set(i, x)`	O(1)	direct indexed slot
append `add(x)`	amortized O(1)	occasional backing-array growth copies elements
insert/remove at index	O(n) worst case	later references shift
`contains`, `indexOf`	O(n)	linear equality checks
iterate	O(n)	visit n elements
copy into a new list	O(n)	n memberships copied

JAVA

```java
static void insertAtFront(List<Integer> values, int value) {
    values.add(0, value);
}
```

If `values` is an `ArrayList` containing `n` elements, the insertion shifts `n` references: O(n). The method signature accepts any `List`, so a production API cannot promise one implementation's performance unless the contract restricts the implementation.

Why append is amortized O(1)

Most appends fill the next free slot. Occasionally capacity is exhausted and the implementation allocates a larger backing array and copies existing references.

For a growth sequence such as capacities 1, 2, 4, 8, ..., the total number of copied slots before reaching `n` is less than `2n`. Spread over `n` appends, average copying per append is constant. This does not mean every append is O(1); one resize can be O(n).

Pre-sizing with `new ArrayList<>(expectedSize)` can reduce resizing when a trustworthy bound is known. It does not change the asymptotic O(n) storage needed for n entries.

LinkedList: nodes plus traversal

`LinkedList` implements both `List` and `Deque`, but it is not a universal faster replacement for `ArrayList`.

Operation	Typical asymptotic cost
`get(i)`	O(n)
find a value	O(n)
add/remove at a known end	O(1)
add/remove through a positioned iterator	O(1) after reaching position
reach a position by index/value	O(n)
iterate	O(n)

The phrase "linked-list insertion is O(1)" omits how the insertion point is obtained. If a method first walks to index `i`, total work is O(n).

For interview queues and stacks, `ArrayDeque` is usually the clearer default. `LinkedList` also carries one node object per element and often has poorer cache locality; exact performance remains workload- and JVM-dependent.

HashMap and HashSet: expected constant-time lookup

`HashMap<K,V>` stores key-value mappings; `HashSet<E>` uses the same hashing idea for unique elements.

Under a sound `hashCode` distribution and ordinary resizing assumptions:

- `put`, `get`, `containsKey`, `remove`: expected O(1);
- insert or lookup can degrade with collisions and implementation state;
- building from `n` entries is expected O(n);
- storing `u` distinct entries uses O(u) logical space.

Do not say "HashMap is guaranteed O(1)." Also do not invent one universal worst-case without naming the Java version, comparable-key conditions, table state, and collision behavior. Expected O(1), with a correctness requirement on `equals` and `hashCode`, is the safe fundamentals answer.

Frequency map cost

JAVA

```java
static Map<Integer, Integer> frequencies(int[] values) {
    Map<Integer, Integer> counts = new HashMap<>();
    for (int value : values) {
        counts.merge(value, 1, Integer::sum);
    }
    return counts;
}
```

Let `n` be array length and `u` the number of distinct values. Expected time is O(n); result space is O(u). The code boxes primitive values/counts. If value range is small and known, an `int[]` frequency table may use less overhead and give worst-case constant indexed updates.

Equality and mutable keys

Hash collections locate a key using its hash and then equality. Equal keys must have equal hash codes. If fields used by `equals/hashCode` change after insertion, lookup and removal may fail because the object no longer maps to the expected bucket. Prefer immutable key state.

Iteration is work too

Iterating a map's entries is at least O(n) in the number of mappings and can depend on implementation capacity. Do not label a method O(k) merely because it returns `k` selected entries if it first scans the whole map.

LinkedHashMap and LinkedHashSet: encounter order

These structures preserve a defined encounter order while retaining hash-based lookup characteristics. Basic lookup remains expected O(1); iteration visits entries in encounter order; linked bookkeeping adds constant-factor memory and update work.

Use them when deterministic encounter order is part of the contract, not as a reflex for every map/set.

TreeMap and TreeSet: sorted order

Tree-based sorted collections maintain elements/keys according to natural order or a comparator.

Operation	Cost
add/put/get/contains/remove	O(log n)
first/last/floor/ceiling style navigation	O(log n)
iterate all entries in order	O(n)
report a range of k entries	O(log n + k) as a useful model

```java
static Integer smallestAtLeast(TreeSet<Integer> values, int target) {
    return values.ceiling(target);
}
```

This query is O(log n). A `HashSet` cannot answer it directly because it provides membership, not sorted navigation.

Comparator consistency matters: if comparison returns zero for values that are not intended to be the same set key, one may replace/suppress the other from the sorted collection's perspective.

ArrayDeque: queue and stack ends

`ArrayDeque` supports adding/removing at both ends, with end operations amortized O(1).

```java
Deque<Integer> queue = new ArrayDeque<>();
queue.offerLast(10);
queue.offerLast(20);
int first = queue.removeFirst();

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
int top = stack.pop();
```

A sequence of `n` enqueue/dequeue or push/pop operations is O(n) total. Searching for an arbitrary value is O(n). ArrayDeque does not permit `null`; that keeps `null` available as an empty-result signal for methods such as `poll` and `peek`.

Avoid legacy `Stack` for new interview code; its historical inheritance and synchronization rarely match the requested abstraction.

PriorityQueue: access the next priority, not a sorted list

Java's default `PriorityQueue` is a min-priority queue.

Operation	Cost
`peek`	O(1)
`offer`	O(log n)
`poll`	O(log n)
find/remove an arbitrary value	O(n)
drain all values by repeated `poll`	O(n log n)

JAVA

```
PriorityQueue<Integer> maximumFirst =
        new PriorityQueue<>(Comparator.reverseOrder());
```

For custom objects, use `Integer.compare`, `Long.compare`, or comparator composition rather than subtraction that can overflow.

JAVA

```
record Job(int priority, long sequence) {}

PriorityQueue<Job> jobs = new PriorityQueue<>(
        Comparator.comparingInt(Job::priority)
                  .thenComparingLong(Job::sequence));
```

Iteration over a `PriorityQueue` is **not** sorted order. Only the head is guaranteed. Poll repeatedly if ordered removal is required, accepting O(n log n) time and mutation of the queue (or copy first).

Sorting, binary search, and conversion

Sorting

- Sorting `n` values is commonly O(n log n), but exact worst-case and auxiliary-space guarantees depend on the Java overload and element type.
- `Collections.sort(list)` and `list.sort(comparator)` mutate the list.
- Copy before sorting when the caller's order must remain unchanged; the copy adds O(n) time and space.

Binary search

`Arrays.binarySearch` or `Collections.binarySearch` is meaningful only when data is sorted under a compatible order. Search is O(log n) on an array or random-access list. On a sequential-access list, traversal can dominate even if comparison count is logarithmic. State the representation.

Conversions and views

- `new HashSet<>(list)` is expected O(n) build time and O(u) entries.
- `new ArrayList<>(set)` is O(n) time and O(n) slots.
- `Arrays.asList(objectArray)` creates a fixed-size list backed by the array; it does not copy elements into an independent resizable list.
- `List.copyOf(source)` creates an unmodifiable list snapshot of the memberships and is O(n) in the ordinary analysis.
- `subList` is a view; changing the backing list structurally can invalidate it.

Worked comparison: duplicate detection

Baseline

JAVA

```java
static boolean hasDuplicateQuadratic(int[] values) {
    for (int left = 0; left < values.length; left++) {
        for (int right = left + 1; right < values.length; right++) {
            if (values[left] == values[right]) return true;
        }
    }
    return false;
}
```

Worst-case time is O(n squared); auxiliary space is O(1).

Hashing trade-off

JAVA

```java
static boolean hasDuplicateExpectedLinear(int[] values) {
    Set<Integer> seen = new HashSet<>();
    for (int value : values) {
        if (!seen.add(value)) return true;
    }
    return false;
}
```

Expected time is O(n); auxiliary space is O(u), up to O(n). The solution also boxes integers. This is not universally "better": tight memory, adversarial behavior, a tiny `n`, or a constrained value range can change the choice.

Sorting trade-off

JAVA

```java
static boolean hasDuplicateBySorting(int[] values) {
    int[] copy = Arrays.copyOf(values, values.length);
    Arrays.sort(copy);
    for (int index = 1; index < copy.length; index++) {
        if (copy[index - 1] == copy[index]) return true;
    }
    return false;
}
```

Time is dominated by sorting, commonly O(n log n). The copy makes the original safe and adds O(n) space. Sorting the input directly can reduce additional result storage but mutates caller data.

Collection selection table

Requirement	Starting choice	Important qualification
indexed sequence, append-heavy	`ArrayList`	middle shifts are O(n)
unique membership	`HashSet`	expected O(1), equality/hash contract
unique + insertion order	`LinkedHashSet`	extra linked bookkeeping
unique + sorted navigation	`TreeSet`	O(log n), comparator semantics
key/value lookup	`HashMap`	expected O(1), mutable keys unsafe
key/value + encounter order	`LinkedHashMap`	order is part of contract
sorted keys/range queries	`TreeMap`	O(log n), comparator semantics
FIFO or LIFO ends	`ArrayDeque`	no `null`; arbitrary search O(n)
repeatedly remove min/max	`PriorityQueue`	iteration is not sorted

WATCH OUT | Common mistakes

- Claiming all collection operations are O(1).
- Choosing by interface name without naming the concrete implementation.
- Saying linked lists make insertion O(1) while ignoring traversal.
- Forgetting `ArrayList.contains` is linear.
- Reporting HashMap worst-case behavior as a universal O(1) guarantee.
- Ignoring equality, hashing, ordering, boxing, or mutation contracts.
- Assuming PriorityQueue iteration is sorted.
- Using subtraction in a comparator.
- Calling `Arrays.asList(intArray)` a list of integers; it is a one-element list whose element is the whole primitive array.
- Forgetting that a copy or conversion is O(n).

Quick check

1. Why is ArrayList append amortized rather than worst-case O(1)?
2. Why can LinkedList insertion still take O(n)?
3. Which qualifier belongs before O(1) HashMap lookup?
4. What is the difference between TreeSet membership and HashSet membership?
5. Why is PriorityQueue iteration not a sorted traversal?
6. When does a range query naturally use `k` in its bound?
7. What correctness requirement makes a mutable HashMap key risky?

Practice

1. **Foundation:** Choose implementations for a resizable list, unique tags, FIFO tasks, and smallest-priority task.
2. **Foundation:** Analyze `for (int x : list) if (list.contains(x)) ...` for an ArrayList.
3. **Interview Core:** Compare three duplicate-detection solutions by time, space, mutation, and guarantees.
4. **Interview Core:** Design a frequency counter that preserves first-seen key order.
5. **Interview Core:** Analyze top-k selection using a heap of size `k`.
6. **Interview Core:** Explain the cost of copying and sorting a list before binary search.
7. **SDE-2 Follow-up:** Choose between HashMap and TreeMap for an API supporting point lookup plus inclusive key ranges.
8. **SDE-2 Follow-up:** Explain what evidence would justify pre-sizing a high-volume HashMap.

RECAP | Chapter summary

Choose collections by semantics first and complexity second. Name the concrete implementation, qualify expected and amortized costs, include traversal and copying, and state equality, order, mutation, and memory assumptions. That is the level of defensible reasoning expected in an SDE-2 interview.

SDE-2 CORE CHAPTER

Chapter 5 - SDE-2 Complexity Reasoning and the Optimization Method

Learning objectives

By the end of this chapter, you should be able to:

- derive time and space complexity from executed work rather than loop appearance;
- distinguish worst-case, expected, amortized, and output-sensitive costs;
- state preconditions, postconditions, loop invariants, and progress measures;
- move from a correct baseline to an optimized algorithm systematically; and
- communicate a complete SDE-2 solution, including trade-offs, proof, tests, and production limits.

WHY IT WORKS | Why this matters at SDE-2

Coding interviews at this level do not reward pattern recall alone. The interviewer is looking for controlled reasoning under incomplete requirements. Can you expose an ambiguity before it becomes a bug? Can you justify discarding part of the search space? Can you separate an algorithmic bound from Java implementation costs? Can you recover when the first approach is too slow?

The same skill appears in production. A service incident might require estimating whether a loop is linear in records, quadratic in tenant pairs, or bounded by output. A design review might hinge on whether memory is O(n) per request or O(n) for the whole process. Complexity vocabulary is useful only when tied to named inputs and an execution model.

> **Learning-path position:** This chapter converts the foundations from Chapters 1-4 into an SDE-2 explanation method. After completing this volume, continue to Number Systems Study Step 03A for numeric representations, overflow boundaries, modular arithmetic, powers, roots, and the mathematical tools used by later DSA books.

WHY IT WORKS | First-principles model

An algorithm transforms inputs satisfying preconditions into an output satisfying postconditions. Correctness has two parts: partial correctness, meaning the promised result holds if the algorithm terminates, and termination, meaning it eventually stops for every valid input.

An invariant is a statement true at a defined program point before and after each iteration or recursive expansion. A progress measure moves toward a bound. Together they turn intuition into a proof. For example, binary search does not work because it "checks the middle." It works because the answer remains inside a maintained interval and each comparison safely removes a region that cannot contain it.

Complexity describes resource growth as input dimensions grow. It ignores many machine constants to compare scalability, but it does not erase them from engineering. First derive the asymptotic model. Then discuss allocations, cache locality, hashing behavior, boxing, and expected data sizes.

> **Specification boundary:** Big-O is a mathematical bound, not a Java or JVM guarantee about elapsed time. Java specifies observable program behavior; it does not promise a particular instruction count, cache behavior, object size, or JIT optimization for a source-level operation.

Core terminology

- **Input dimension:** independent size variable such as `n` records, `m` queries, or `V` vertices and `E` edges.
- **Big-O:** asymptotic upper bound.
- **Big-Omega:** asymptotic lower bound.
- **Big-Theta:** matching upper and lower bound.
- **Worst case:** maximum cost among inputs of a given size.
- **Expected case:** average over a stated randomness or input distribution.
- **Amortized cost:** total cost of an operation sequence divided across that sequence, without assuming random inputs.
- **Auxiliary space:** extra working storage excluding input and usually output.
- **Output-sensitive:** cost includes the size of the result, such as O(n + k) for k matches.
- **Invariant:** property preserved at a chosen control point.
- **Progress measure:** quantity that moves monotonically toward termination.
- **Baseline:** simplest clearly correct approach used to expose structure and constraints.

Detailed mechanics

A defensible complexity analysis

Start by naming dimensions. If one loop reads `users` and another reads `orders`, call the sizes `u` and `o`; do not collapse them into `n` unless the relationship is known. Count a meaningful dominant operation, form a sum, and simplify afterward.

Sequential phases add. Sorting and then scanning is O(n log n + n), which simplifies to O(n log n). Independent nested loops multiply: comparing every user with every role is O(ur). Dependent loops require a sum. This loop is O(n), not O(n squared), because `right` only advances:

JAVA

```java
for (int left = 0, right = 0; right < values.length; right++) {
    while (left <= right && windowIsInvalid(left, right)) {
        left++;
    }
}
```

Across the whole execution, each pointer moves at most n times. Aggregate pointer movement is often the right unit for windows, monotonic structures, and union-find analyses.

Halving a remaining range produces logarithmic depth: after k halvings, size is approximately n / 2^k, so reaching one requires k near log2(n). A balanced divide-and-conquer algorithm that does linear work at each of log n levels is O(n log n). Recursive analysis must include both number of calls and work per call; `T(n) = 2T(n/2) + O(n)` differs from `T(n) = T(n/2) + O(1)`.

Space analysis separates several categories:

Category	Question	Example
Input	What storage already exists?	Supplied `int[]`
Auxiliary	What new working state is needed?	Hash map of n entries
Call stack	How deep can recursion become?	O(h) tree height
Output	How large must the result be?	O(k) returned matches
Hidden library work	Does an API copy or box?	Sorting objects, `substring`, streams

An "in-place" recursive algorithm may still use O(n) call-stack space. Returning all pairs cannot use less than O(k) output storage if k pairs are materialized.

Worst, expected, and amortized claims

State the qualifier. Hash-table operations are commonly expected O(1) under suitable hashing and load, not an unconditional mathematical constant for every possible collision pattern. Dynamic-array append is amortized O(1): rare O(n) resize copies are paid for across many cheap appends. Randomized quickselect is expected O(n) with an appropriate pivot strategy but has a quadratic worst case.

Do not call amortized analysis "average case." Amortization holds across any operation sequence covered by its potential or aggregate proof; expected analysis depends on probability.

The SDE-2 control loop

Use the following repeatable sequence:

1. **Contract:** restate inputs, outputs, mutability, null policy, duplicates, ordering, ranges, and failure behavior.
2. **Examples:** create a normal case and an adversarial boundary. Calculate expected output manually.
3. **Baseline:** describe a correct direct solution and its cost. This establishes a fallback.
4. **Constraint pressure:** compare baseline cost with input bounds. Name the repeated or unnecessary work.
5. **Representation:** choose the state needed to avoid that work: hash map, pointer boundary, prefix state, heap, graph frontier, or DP table.
6. **Invariant:** say what every variable or data structure means at a stable point.
7. **Progress:** show why each iteration or recursive call approaches completion.
8. **Implementation:** use names that encode the proof and keep the main path visible.
9. **Trace:** execute the code on a boundary and a representative case.
10. **Close:** state time, auxiliary space, output space, mutation, and relevant alternatives.

This is not ceremony. Each step catches a different failure class. The contract catches wrong problems, the baseline catches unjustified optimization, the invariant catches logic errors, and the trace catches boundary defects.

Invariant proof template

For a loop, answer three questions:

- **Initialization:** why is the invariant true before the first iteration?
- **Maintenance:** assuming it is true, why does one iteration preserve it?
- **Termination:** when the loop stops, how does the invariant imply the postcondition?

Then give a variant or progress measure that is bounded and changes every continuing iteration. This proof style works for partitioning, two pointers, BFS layers, heap selection, and DP fill order.

Binary search as a reasoning template

Binary search applies to a monotone predicate, not only to equality in a sorted array. Define a search interval consistently. A half-open interval `[low, high)` is often easiest because its size is `high - low`, empty means `low == high`, and `high` can be `n` for insertion at the end.

For lower bound, maintain:

- every index before `low` has value less than target;
- every index at or after `high` has value at least target; and
- the first qualifying index remains in `[low, high)`.

Use `mid = low + (high - low) / 2` and update one boundary to exclude `mid`; otherwise the interval may not shrink.

TRACE IT | Worked Java example

The following Java 21 program returns the first index whose value is at least the target. It returns `values.length` when no element qualifies.

JAVA

```java
import java.util.Arrays;

public final class LowerBound {
    public static int lowerBound(int[] values, int target) {
        int low = 0;
        int high = values.length;

        while (low < high) {
            int mid = low + (high - low) / 2;
            if (values[mid] < target) {
                low = mid + 1;
            } else {
                high = mid;
            }
        }
        return low;
    }

    private static void requireSorted(int[] values) {
        for (int i = 1; i < values.length; i++) {
            if (values[i - 1] > values[i]) {
                throw new IllegalArgumentException("array must be sorted");
            }
        }
    }

    public static void main(String[] args) {
        int[] values = {1, 3, 3, 7, 9};
        requireSorted(values);
        for (int target : new int[] {0, 3, 4, 10}) {
            System.out.printf("target=%d index=%d%n",
                    target, lowerBound(values, target));
        }
        System.out.println(Arrays.toString(values));
    }
}
```

The sortedness check is useful at an external boundary but would change a single search from O(log n) to O(n). In an interview, state whether sorted input is a trusted precondition instead of silently validating it.

TRACE IT | Execution or memory walkthrough

Trace `lowerBound([1, 3, 3, 7, 9], 3)`:

Step	`low`	`high`	`mid`	`values[mid]`	Update
Start	0	5	-	-	Candidate interval `[0, 5)`
1	0	5	2	3	`high = 2`
2	0	2	1	3	`high = 1`
3	0	1	0	1	`low = 1`
End	1	1	-	-	Return 1

At each start, indices below `low` are proven too small, indices at or above `high` are proven qualifying, and the boundary lies between them. Each iteration strictly reduces `high - low`. At termination, no candidate interval remains; `low` is the unique partition point.

The method stores four primitive locals and uses no recursion, so auxiliary space is O(1). It does not mutate the input. The JVM may place locals in registers after compilation, but that does not change the source-level space analysis.

Complexity and performance

Pattern	Time	Auxiliary space	Important qualifier
Full scan	O(n)	O(1)	May stop early, worst case remains n
Compare all pairs	O(n^2)	O(1)	Output may itself be quadratic
Sort then scan	O(n log n)	Depends on sort	Mutation and stability matter
Binary search	O(log n)	O(1) iterative	Requires monotone search space
Hash lookup sequence	Expected O(n)	O(n)	Hash quality and boxing matter
Balanced recursion	Often O(log n) depth	O(log n) stack	Skew can make depth O(n)
Dynamic array append	Amortized O(1)	Capacity overhead	Individual resize is O(n)

`lowerBound` is O(log n) time and O(1) auxiliary space when sortedness is a precondition. Calling `requireSorted` first makes the combined operation O(n). For one query on untrusted data, a linear scan may be simpler. For many queries, validating once and reusing the ordered representation can be worthwhile.

Constants re-enter after the asymptotic choice. A linear scan over a small contiguous primitive array can beat a more elaborate structure. Benchmark representative workloads rather than treating the notation as a stopwatch.

> **HotSpot note:** HotSpot may inline methods, eliminate range checks, vectorize loops, and compile hot branches using profile data. These optimizations can change constants but not the algorithm's asymptotic operation count or correctness proof.

WATCH OUT | Edge cases and common mistakes

- Naming only `n` when the algorithm depends independently on two inputs.
- Multiplying every pair of nested loops without checking total pointer movement.
- Dropping output space or recursive stack space from the analysis.
- Claiming hash operations are worst-case O(1) without qualification.
- Confusing expected with amortized complexity.
- Optimizing before clarifying whether mutation, sorting, or extra memory is allowed.
- Choosing a pattern from keywords without proving its invariant.
- Mixing closed `[low, high]` and half-open `[low, high)` binary-search rules.
- Computing `(low + high) / 2` when indexes or search values could overflow.
- Updating `low = mid` in a loop where `mid` can equal `low`, causing nontermination.
- Giving average-case complexity when the interviewer asked for a service limit or worst-case bound.
- Hiding assumptions about integer overflow, Unicode, duplicates, or invalid input.

Production engineering notes

Production inputs have distributions, limits, and adversaries. Record all three. Hash-flooding, enormous result sets, recursion depth, and a caller-controlled sort key can turn a textbook choice into a reliability problem. Put hard bounds around memory and output, use `long` for counts that may exceed `int`, and fail before partial mutation when possible.

Complexity is end-to-end. Reading n rows through an ORM may perform n extra queries; an O(n) Java loop can sit on top of O(n) network round trips. A theoretically better algorithm can lose if it allocates millions of wrappers or destroys locality. Instrument the actual bottleneck and preserve a clear correctness model while optimizing.

During review, demand qualifiers: n means what, hash time under what assumption, recursion depth under which shape, and whether output is included. This precision transfers directly from interviews to capacity planning.

TRY IT | Interview questions and model answers

Are two nested loops always O(n squared)?

No. Derive total executions. In a sliding window, two nested-looking pointer loops may advance each pointer at most n times, yielding O(n). Independent full-range loops do produce a product.

What is the difference between expected and amortized O(1)?

Expected O(1) relies on a probability model, such as suitable hash distribution. Amortized O(1) bounds the average cost across an operation sequence, such as dynamic-array appends including occasional resizes, without assuming random inputs.

How do you prove binary search correct?

Define a monotone predicate and an interval invariant that retains the boundary or answer. Show initialization, show each comparison discards only impossible values, show the interval shrinks, and use the termination condition to derive the returned boundary.

Why start with a brute-force baseline?

It validates understanding, gives a correctness oracle for small tests, identifies repeated work, and provides a fallback. The optimized solution should be motivated by constraints, not pattern guessing.

What space should be reported?

State auxiliary working space, recursion stack, and output space separately. Also mention whether input is mutated. This avoids calling a recursive or output-heavy method "constant space."

When can O(n) be worse than O(n log n)?

Asymptotic notation describes growth, not every concrete runtime. The O(n) approach may have much larger constants, random memory access, boxing, or remote operations. For sufficiently large n its growth is better, but production decisions need representative measurements.

TRY IT | Exercises

1. Analyze a loop with two pointers that each only move forward. Write the aggregate sum that proves O(n).
2. Implement upper bound, returning the first index whose value is greater than a target, using a half-open interval.
3. Analyze `T(n) = 2T(n/2) + n` with a recursion tree and account for stack depth separately.
4. Give a potential or aggregate argument for amortized dynamic-array append.
5. Take a quadratic duplicate-detection baseline and derive hashing and sorting alternatives, including mutation and worst-case qualifiers.
6. Write preconditions, postconditions, an invariant, and a progress measure for stable in-place compaction.
7. Estimate memory for one million boxed map entries, then explain why asymptotic O(n) alone is insufficient for a capacity decision.

RECAP | Chapter summary

Strong problem solving begins with a contract and ends with a proof, a cost model, and tests. Name independent input dimensions, count total work, qualify expected and amortized claims, and separate auxiliary, stack, and output space. Use a correct baseline to expose repeated work. Then choose state, state an invariant, prove progress, and trace boundaries. Binary search is the canonical example: its power comes from a monotone predicate and a shrinking candidate interval, not from memorized syntax.

TRY IT | Revision checklist

- [ ] I name every independent input dimension before analyzing cost.
- [ ] I sum dependent work and multiply only independent repeated work.
- [ ] I distinguish O, Omega, Theta, worst, expected, and amortized bounds.
- [ ] I report auxiliary, call-stack, output, and mutation costs separately.
- [ ] I clarify the contract and produce a correct baseline before optimizing.
- [ ] I state invariants with initialization, maintenance, and termination.
- [ ] I identify a bounded progress measure.
- [ ] I can implement lower and upper bound without mixing interval conventions.
- [ ] I discuss constants and production constraints only after the asymptotic model is sound.

SDE-2 CORE CHAPTER

Chapter 6 - Collection Complexity and Selection Matrix

Complexities below describe common OpenJDK implementations for ordinary workloads, not interface-level guarantees unless stated. Hash-table operations are expected-time claims under an adequate hash distribution. Adversarial keys, resizing, comparator cost, allocation, cache locality, and concurrency can dominate a real service.

List implementations

Operation	`ArrayList`	`LinkedList`	Engineering note
indexed `get` / `set`	O(1)	O(n)	Array locality usually favors `ArrayList`
append	amortized O(1)	O(1)	Array resize occasionally copies elements
prepend	O(n)	O(1)	`ArrayDeque` is usually the better queue
insert/remove by index	O(n) shift	O(n) traversal, O(1) relink	Traversal often dominates linked-list edits
search by value	O(n)	O(n)	Equality cost is part of the constant
iteration	O(n)	O(n)	Pointer chasing can hurt locality
memory per element	reference plus spare capacity	node object plus links	Measure retained size for large collections

Default choice: `ArrayList`. Choose `LinkedList` only when its list-plus-deque contract and a measured mutation pattern justify node overhead. Even frequent insertion is not sufficient if finding the insertion point is linear.

`CopyOnWriteArrayList` makes traversal stable and unsynchronized for readers by copying the backing array on every structural mutation. It fits small, read-mostly listener/configuration sets; it is disastrous for frequent writes or large lists.

Set and map implementations

Type	Lookup / update	Ordering	Null policy	Representative use
`HashMap`	expected O(1)	unspecified	one null key, null values	general mutable map
`LinkedHashMap`	expected O(1)	insertion or access order	like `HashMap`	predictable traversal, bounded LRU building block
`TreeMap`	O(log n)	comparator/natural sorted	null key generally rejected by natural ordering	range queries, nearest-key navigation
`EnumMap`	O(1) array-like	enum declaration order	null key rejected	dense enum-keyed state
`IdentityHashMap`	expected O(1)	unspecified	permits null	graph/topology algorithms using identity semantics
`ConcurrentHashMap`	expected O(1)	unspecified, weakly consistent traversal	null keys/values rejected	shared concurrent lookup/update
immutable `Map.of` / `Map.copyOf`	implementation-dependent efficient lookup	unspecified	null rejected	fixed snapshots and safe sharing

Corresponding sets are commonly backed by maps or tree structures:

Type	Membership	Ordering	Notes
`HashSet`	expected O(1)	unspecified	mutable elements can become unfindable
`LinkedHashSet`	expected O(1)	insertion order	extra link metadata
`TreeSet`	O(log n)	sorted	comparator equality controls uniqueness
`EnumSet`	O(1) bit-vector-like	enum order	exceptionally compact for enum domains
`CopyOnWriteArraySet`	O(n) lookup, O(n) copy on write	insertion-like snapshot order	very small read-mostly sets
`ConcurrentHashMap.newKeySet()`	expected O(1)	weakly consistent	scalable concurrent membership set

Important `HashMap` facts:

- Capacity and load factor affect resizing, memory, and traversal cost.

- Current OpenJDK `HashMap` implementations may transform a collision-heavy bin into a tree when thresholds and table capacity permit. That is an implementation strategy, not permission to use poor keys.
- `computeIfAbsent` is not a general exactly-once side-effect facility. Read each map's contract, keep mapping functions short, and avoid recursive structural updates.
- Mutating an equality/hash field after insertion violates the lookup assumption even though the entry still occupies a bucket.
- Iteration order can look stable in a small experiment and still have no contract.

Queues, deques, and heaps

Type	Add/remove	Head access	Capacity / blocking	Use
`ArrayDeque`	amortized O(1) at ends	O(1)	unbounded, nonblocking	default stack, queue, deque
`PriorityQueue`	O(log n) add/remove	O(1) peek	unbounded, nonblocking	next minimum/maximum by comparator
`ConcurrentLinkedQueue`	expected O(1)	typically O(1); may traverse removed nodes	unbounded, nonblocking	scalable multi-producer/consumer queue
`ArrayBlockingQueue`	O(1)	O(1)	bounded, blocking	fixed-capacity back pressure
`LinkedBlockingQueue`	O(1)	O(1)	optionally bounded, blocking	producer/consumer, usually explicit bound
`PriorityBlockingQueue`	O(log n)	O(1)	unbounded, blocking take	concurrent priority scheduling without capacity control
`SynchronousQueue`	handoff	none stored	zero capacity, blocking	direct producer-consumer rendezvous
`DelayQueue`	O(log n)	eligible head	unbounded	delayed/expiry work

Use `offer`, `poll`, and `peek` when absence/capacity is expected and should be represented by a result. Use `add`, `remove`, and `element` when failure should be exceptional. Blocking queues add `put` and `take`; timed variants let cancellation and service-level policy participate.

`PriorityQueue` guarantees that `peek`/`remove` exposes a least element according to its ordering. Its iterator is not sorted. To emit sorted order, repeatedly remove elements from a copy, or sort a collection explicitly.

Sorted and navigable operations

`NavigableMap` and `NavigableSet` support relative queries in O(log n) for tree implementations:

- `lower`: greatest element strictly less than the key.
- `floor`: greatest element less than or equal to the key.
- `ceiling`: least element greater than or equal to the key.
- `higher`: least element strictly greater than the key.
- `subMap`, `headMap`, and `tailMap`: usually backed range views.

Backed views enforce their range. An insertion outside the view range fails. Changes through either the view or the source are visible through the other, subject to the collection's synchronization and iterator contracts.

Views, wrappers, and copies

These APIs have deliberately different ownership semantics:

API	Structural mutability	Backing relationship	Null handling
`Arrays.asList(array)`	fixed size; `set` allowed	backed by array	permits null
`list.subList(a, b)`	generally mutable within range	backed by list	source policy
`Collections.unmodifiableList(list)`	wrapper rejects mutation	reflects backing-list changes	source policy
`List.copyOf(source)`	unmodifiable	snapshot-like shallow copy/reuse	rejects null
`List.of(values...)`	unmodifiable	independent fixed value	rejects null
`stream.toList()`	unmodifiable	result collection	may contain null elements produced by stream
`Collectors.toList()`	no mutability/type guarantee	new result	collector behavior

Unmodifiable is not deeply immutable. If elements are mutable, their state can change. A defensive copy protects collection structure only; copy elements or use immutable value types when deeper isolation is required.

Iterator consistency

Iterator labels describe detection and visibility, not thread safety of arbitrary compound actions:

- Fail-fast iterators on many ordinary collections may throw `ConcurrentModificationException` after an uncoordinated structural change. Detection is best-effort, not a synchronization mechanism.
- Snapshot iterators, such as those of copy-on-write collections, traverse a stable historical array and do not see later writes.
- Weakly consistent iterators on concurrent collections tolerate concurrent updates and may reflect some updates without throwing. They do not freeze a transactionally consistent snapshot.

Use external locking when multiple operations must preserve one invariant and the collection does not supply an atomic compound method. `if (!map.containsKey(k)) map.put(k, v)` is a race on a shared map; use `putIfAbsent`, `compute`, or explicit coordination as the required semantics dictate.

Stream and collector cost reminders

A stream pipeline is not a collection. It describes one traversal and is normally single-use. Complexity is the sum of traversal and operations:

Operation	Typical cost	Important qualification
`filter`, `map`	O(n)	function cost and boxing may dominate
`distinct`	expected O(n)	retains seen values; ordered mode can cost more
`sorted`	O(n log n)	buffers elements; comparator cost matters
`limit`	passes at most k downstream	upstream filters or stateful stages can still perform O(n) work; ordered parallel pipelines may coordinate heavily
`findFirst`	short-circuiting	encounter order constrains parallel execution
grouping	expected O(n)	retains keys and downstream accumulation state
`toMap`	expected O(n)	merge policy required when duplicate output keys are possible

Collector characteristics such as `CONCURRENT`, `UNORDERED`, and `IDENTITY_FINISH` are promises to the reduction framework. A mutable result container alone does not make a collector concurrent.

Selection questions

Choose a collection by answering in order:

1. What semantics are required: sequence, uniqueness, association, priority, range navigation, or handoff?
2. Is order absent, insertion-based, access-based, sorted, or merely head-priority?
3. Which operations dominate, and what are their input sizes?
4. Is mutation local, externally synchronized, or concurrent?
5. Must a compound invariant span several operations?
6. Are nulls valid domain values, and can the chosen type represent them?
7. Who owns the collection, and should the API expose a view, wrapper, or copy?
8. What bounds memory: maximum size, admission control, expiry, or back pressure?
9. Does iteration need a snapshot, weak consistency, or strict external coordination?
10. Have representative allocation, locality, contention, and tail latency been measured?

Rapid decision matrix

The strongest interview answer states the contract first, compares two plausible choices, names the dominant workload operation, and closes with ownership, concurrency, and measurement implications.

Requirement	Start with	Reconsider when
general indexed sequence	`ArrayList`	frequent measured front edits or immutable persistent structure needed
stack / FIFO / double-ended work	`ArrayDeque`	cross-thread blocking or bounded capacity required
general key lookup	`HashMap`	sorted/range operations, stable order, enum keys, or concurrency required
predictable insertion traversal	`LinkedHashMap`	sorted key semantics required
sorted map and nearest-key query	`TreeMap`	hash lookup is sufficient and ordering cost is wasted
enum membership	`EnumSet`	domain is not an enum
minimum/maximum scheduling	`PriorityQueue`	full sorted iteration or bounded blocking is required
shared concurrent map	`ConcurrentHashMap`	one invariant spans external state or several keys
bounded producer/consumer	`ArrayBlockingQueue`	transfer semantics, priority, or different throughput characteristics matter
fixed public result	`List.copyOf` / `Map.copyOf`	caller intentionally needs a live view
small read-mostly listener set	copy-on-write collection	writes or collection size grow

SDE-2 CORE CHAPTER

Chapter 7 - Complexity Practice Lab

How to use this lab

Attempt each item before reading the solutions chapter. For every analysis answer, write:

1. input dimensions;
2. time bound and case;
3. auxiliary/output space;
4. assumptions about APIs or data;
5. one sentence of proof.

The difficulty labels describe the reasoning expected, not the amount of code.

A. Knowledge check

1. **Foundation:** What does Big-O describe, and what does it not describe?
2. **Foundation:** Why must two independent array lengths be named `n` and `m`?
3. **Foundation:** Distinguish input space, auxiliary space, and output space.
4. **Foundation:** Why are two consecutive O(n) loops still O(n)?
5. **Foundation:** What code shape commonly creates O(log n) work?
6. **Foundation:** Why is array indexing O(1)?
7. **Foundation:** When does a pair-enumeration loop become O(n squared)?
8. **Foundation:** What does "worst case" mean for linear search?
9. **Foundation:** Why should constants be dropped only after deriving a count?
10. **Interview Core:** Distinguish expected and amortized complexity.
11. **Interview Core:** What makes a complexity bound output-sensitive?
12. **Interview Core:** Why can a nested-looking two-pointer loop be O(n)?
13. **Interview Core:** Why is recursive Fibonacci exponential time but linear stack space?
14. **Interview Core:** What is the most accurate stack-space dimension for tree DFS?
15. **Interview Core:** Why is `HashMap.get` not guaranteed O(1) in every circumstance?
16. **Interview Core:** Why can LinkedList insertion still require O(n) total time?
17. **Interview Core:** What makes ArrayList append amortized O(1)?
18. **Interview Core:** Why is PriorityQueue iteration not a sorted traversal?
19. **SDE-2 Follow-up:** When is an O(n log n) solution preferable to expected O(n)?
20. **SDE-2 Follow-up:** Why can the same asymptotic complexity have very different production behavior?

B. Predict the count and derive the bound

For each snippet, give an exact body count when practical and then simplify.

B1. Fixed access

JAVA

```java
int answer = values[0] + values[values.length - 1];
```

B2. Sequential phases

JAVA

```java
for (int value : first) consume(value);
for (int value : second) consume(value);
```

B3. Square grid

JAVA

```java
for (int row = 0; row < n; row++)
    for (int column = 0; column < n; column++)
        visit(row, column);
```

B4. Triangular pairs

JAVA

```java
for (int left = 0; left < n; left++)
    for (int right = left + 1; right < n; right++)
        compare(left, right);
```

B5. Halving

JAVA

```java
for (int value = n; value > 1; value /= 2) work();
```

B6. Doubling plus full scan

JAVA

```java
for (int size = 1; size < n; size *= 2)
    for (int index = 0; index < n; index++)
        work();
```

B7. Geometric inner work

JAVA

```java
for (int size = 1; size <= n; size *= 2)
    for (int index = 0; index < size; index++)
        work();
```

B8. Forward-only pointers

JAVA

```java
int left = 0;
for (int right = 0; right < n; right++) {
    while (left < right && tooLarge(left, right)) left++;
}
```

Assume `tooLarge` is O(1).

B9. ArrayList membership

JAVA

```java
for (int value : values) {
    if (list.contains(value)) matches++;
}
```

Both `values` and `list` contain `n` elements and `list` is an ArrayList.

B10. HashSet membership

Repeat B9 when `list` is replaced by a soundly hashed HashSet.

B11. Sorting and scanning

JAVA

```java
Arrays.sort(values);
for (int value : values) consume(value);
```

State the ordinary primitive-array interview model and qualify it.

B12. Repeated concatenation

JAVA

```java
String result = "";
for (char character : characters) result += character;
```

Let the final string length be `n`.

B13. Jagged matrix

JAVA

```java
for (int[] row : matrix)
    for (int value : row)
        consume(value);
```

Use `r` rows and `c_i` for each row's length.

B14. Recursive countdown

JAVA

```java
static void countDown(int n) {
    if (n == 0) return;
    countDown(n - 1);
}
```

B15. Binary recursion

JAVA

```java
static void enumerate(int remaining) {
    if (remaining == 0) return;
    enumerate(remaining - 1);
    enumerate(remaining - 1);
}
```

B16. Output matches

JAVA

```java
List<Integer> result = new ArrayList<>();
for (int value : values)
    if (accept(value)) result.add(value);
```

Assume `accept` is O(1), input length `n`, and `k` matches.

B17. Queue drain

JAVA

```java
while (!queue.isEmpty()) consume(queue.removeFirst());
```

The queue is an ArrayDeque initially containing `n` entries.

B18. Heap drain

JAVA

```java
while (!heap.isEmpty()) consume(heap.remove());
```

The PriorityQueue initially contains `n` entries.

B19. Tree range

JAVA

```java
for (int value : treeSet.subSet(low, true, high, true)) consume(value);
```

Let `k` elements lie in the range.

B20. Many test cases

Test case `i` contains `n_i` values and is scanned once. Express total work for `t` test cases without assuming every case has the same size.

C. Debug the analysis

For each claim, explain the error and write a corrected claim.

1. **Foundation:** "This method has three loops, so it is O(n cubed)." The loops are consecutive.
2. **Foundation:** "Array access is O(n) because the array has n elements."
3. **Foundation:** "The loop always returns early, so worst-case time is O(1)."
4. **Foundation:** "O(n) means exactly n machine instructions."
5. **Foundation:** "The two arrays have total complexity O(n)" even though lengths are independent.
6. **Interview Core:** "Every nested loop is O(n squared)." Both pointers only move forward.
7. **Interview Core:** "HashMap lookup is guaranteed O(1)."
8. **Interview Core:** "LinkedList insertion is O(1), so insert at index n/2 is O(1)."
9. **Interview Core:** "PriorityQueue iteration prints values in priority order."
10. **Interview Core:** "A recursive method uses O(1) space because it declares no arrays."
11. **Interview Core:** "Fibonacci makes O(2^n) calls, so stack space is O(2^n)."
12. **Interview Core:** "`new ArrayList<>(list)` is O(1) because it is one constructor call."
13. **Interview Core:** "Returning a copied array uses O(1) space if output is excluded," with no mention of result storage.
14. **SDE-2 Follow-up:** "O(n) is always faster than O(n log n)."
15. **SDE-2 Follow-up:** "A benchmark proves this method has O(n) complexity."

D. Small coding and analysis tasks

1. **Foundation:** Write linear search and return both the index and inspection count.
2. **Foundation:** Write a method that visits a rectangular matrix and reports total cells.
3. **Foundation:** Write a repeated-halving method that returns the number of steps.
4. **Interview Core:** Implement duplicate detection using nested loops and using HashSet. Compare contracts.
5. **Interview Core:** Implement a frequency map and state bounds using n and u.
6. **Interview Core:** Reverse an array in place, then implement a non-mutating version.
7. **Interview Core:** Implement a queue and stack using ArrayDeque with consistent method families.
8. **Interview Core:** Keep the largest k integers from a stream using a min-heap of size at most k.
9. **Interview Core:** Implement an O(n) two-pointer scan and prove the aggregate movement bound.
10. **Interview Core:** Implement a jagged-matrix sum and express cost using total cells.
11. **SDE-2 Follow-up:** Refactor repeated string concatenation to StringBuilder and define the input dimension as total characters.
12. **SDE-2 Follow-up:** Design a duplicate-check API that preserves caller input and offers deterministic worst-case behavior; discuss trade-offs.

E. Interview follow-ups

1. Why might you accept O(n log n) sorting instead of expected O(n) hashing?
2. What changes if keys are adversarial or mutable?
3. How would you express graph traversal complexity, and why are both V and E needed?
4. A two-pointer method has a while loop inside a for loop. Prove or refute O(n squared).
5. How does returning all matches change a method that previously returned only the first match?
6. What is amortization, and does it guarantee every request is cheap?
7. When should a copied input be counted in space complexity?
8. How does recursion depth differ on balanced and skewed trees?
9. What assumptions support an expected-case claim?
10. What would a benchmark reveal that Big-O does not, and what would it fail to prove?
11. How do mutation, memory, and deterministic guarantees influence collection choice?
12. Given an O(n squared) baseline, how would you communicate the path to an O(n log n) or expected O(n) solution?

F. Cumulative assessments

Assessment 1: Foundations

Analyze five snippets: fixed access, full scan, repeated halving, two consecutive scans, and pair enumeration. Give one proof sentence each.

Assessment 2: Java cost awareness

Compare ArrayList, HashSet, TreeSet, ArrayDeque, and PriorityQueue for lookup, order, end operations, and priority removal. Include qualifiers.

Assessment 3: Space and ownership

Compare an in-place array transformation, a defensive copy, a recursive traversal, and an output list. Separate auxiliary and output space.

Assessment 4: Hidden work

Find all non-constant work in a method that copies input, sorts it, repeatedly calls `contains` on a list, builds a string, and returns selected values. Rewrite it with clear assumptions.

Assessment 5: SDE-2 explanation

In five minutes, explain a baseline, identify its bottleneck, propose a better data structure or invariant, prove time/space, and state one trade-off plus one test that could falsify your reasoning.

Final readiness assessment

You are ready to continue to Number Systems and the DSA pattern books when you can do all of the following without a reference sheet:

- derive, not guess, bounds for common loop shapes;
- distinguish sequential, independent nested, dependent nested, and aggregate pointer movement;
- state best/worst/expected/amortized qualifiers correctly;
- separate input, auxiliary, output, and recursion-stack space;
- choose core Java collections by semantics and qualified cost;
- include sorting, copying, string construction, boxing, and output work;
- explain one optimization from baseline through proof and trade-off;
- avoid using wall-clock timing as proof of Big-O.

If two or more items are weak, revisit the relevant chapter and rerun `ComplexityExamples.java` before advancing.

SDE-2 CORE CHAPTER

Chapter 8 - Complexity Practice Solutions

Use these solutions after a complete attempt. Equivalent bounds are acceptable when assumptions are explicit and the proof is sound.

How to review a solution

Do not compare only the final notation. For each answer, verify five parts:

1. **Dimension:** Did you name every independent input size rather than forcing everything into n?
2. **Execution:** Did you count reached work, including helper methods, library calls, copies, and generated output?
3. **Qualifier:** Did you label best, worst, expected, or amortized behavior when the distinction changes the claim?
4. **Space contract:** Did you separate auxiliary work, recursion depth, returned output, and mutation of caller data?
5. **Proof:** Can you justify the bound with a count, sum, recurrence, shrinking argument, or aggregate movement invariant?

If your final bound matches but the proof or qualifier is missing, mark the item partially correct. SDE-2 readiness means another engineer can audit the reasoning, not merely recognize the notation.

Keep an error log with four columns: snippet, incorrect assumption, corrected rule, and one new test. Reattempt every missed item after at least one day; delayed retrieval is more useful than rereading the answer immediately.

A. Knowledge-check solutions

1 Big-O is an asymptotic upper bound on growth after constant factors; it is not elapsed time or an exact instruction count.

2 Independent inputs can grow separately. O(n + m) preserves that fact; replacing both by `n` can hide the dominant dimension.

3 Input space belongs to supplied data, auxiliary space is temporary working storage, and output space holds the required result.

4 Their counts add: `n + n = 2n`, which simplifies to O(n). Only independent nested repetitions multiply.

5 Repeatedly halving or doubling a bounded value commonly creates logarithmic iterations.

6 The address of an indexed array slot is computed directly from base, index, and fixed element layout; the program does not scan preceding entries.

7 When all or a constant fraction of unordered/ordered pairs are visited, the count is proportional to `n(n − 1)/2` or `n^2`.

8 The target is last or absent, so all `n` elements are inspected.

9 The unsimplified count shows whether work adds, multiplies, or depends on another dimension; dropping too soon invites a false bound.

10 Expected cost averages over a probability model. Amortized cost averages over an operation sequence even without random input.

11 The bound contains output size `k`, such as O(n + k) time or O(k) result space.

12 If each pointer advances only and at most `n` times over the whole execution, total movement is at most a constant multiple of `n`.

13 The call tree contains exponentially many calls over time, but only one root-to-leaf chain of O(n) frames is active at once.

14 O(h), where `h` is tree height; specialize to O(log n) only with a balance guarantee and O(n) for a worst-case skew.

15 Hash distribution, collisions, resizing, key contracts, and implementation state affect cost. Expected O(1) under sound hashing is the suitable basic claim.

16 Reaching an index or finding a value can require a linear traversal before the link update.

17 Occasional O(n) backing-array copies are spread across many cheap appends, producing constant average cost per append over the sequence.

18 A heap guarantees only the head. Its internal array maintains a partial order, not globally sorted iteration order.

19 Sorting can provide deterministic bounds, ordered output, lower representation overhead, or better locality, and may avoid reliance on hashing/equality.

20 Constants, allocation, locality, input distribution, JVM optimization, APIs, and hardware differ. Asymptotic growth intentionally abstracts these effects.

B. Count-and-bound solutions

1 **B1:** Two indexed reads and one addition: O(1) time, O(1) auxiliary space, assuming a nonempty array.

2 **B2:** `n + m` visits for independent lengths: O(n + m) time and O(1) auxiliary space if `consume` is O(1).

3 **B3:** Exactly `n^2` visits: Theta(n squared) time and O(1) loop storage.

4 **B4:** `n(n − 1)/2` comparisons: Theta(n squared) time, O(1) auxiliary space.

5 **B5:** Floor/ceiling details depend on `n`, but the body runs about `log2(n)` times: O(log n).

6 **B6:** About `log2(n)` outer iterations, each with `n` inner visits: O(n log n).

7 **B7:** `1 + 2 + 4 + ...` up to `n`, less than `2n` for power-of-two framing: O(n), not O(n log n).

8 **B8:** `right` advances `n` times and `left` at most `n` times: O(n) control work by aggregate analysis.

9 **B9:** `n` queries times O(n) linear ArrayList membership: O(n squared) worst-case.

10 **B10:** Expected O(n) total with expected O(1) soundly hashed membership. Space for the existing set is not newly allocated by the loop.

11 **B11:** Sorting dominates the scan, ordinarily O(n log n) time for the relevant primitive overload plus O(n) scan. Exact worst-case and auxiliary-space details should name the overload/JDK contract.

12 **B12:** Each immutable concatenation can copy the growing prefix. `1 + 2 + ... + n` character work is O(n squared); intermediate strings also create allocation pressure. StringBuilder makes construction O(n) in appended characters under the ordinary model.

13 **B13:** Time is Theta(sum from i=1 to r of `c_i`) if each cell is consumed once. The rectangular shorthand O(rows times columns) is inappropriate without equal row lengths.

14 **B14:** `n + 1` calls: O(n) time and O(n) call-stack space.

15 **B15:** The recurrence is `T(n) = 2T(n − 1) + O(1)`, so time is O(2^n) as a simple tight-order description; maximum stack depth is O(n).

16 **B16:** O(n) time, O(1) auxiliary working variables, and O(k) result space. If result storage is included in space, report O(k).

17 **B17:** `n` amortized O(1) end removals: O(n) total. The existing queue occupies O(n) input/state space; loop variables are O(1).

18 **B18:** Removal sizes descend from n; each costs O(log currentSize), totaling O(n log n). The queue is mutated.

19 **B19:** A useful sorted-range model is O(log n + k): locate a boundary then visit `k` entries. Exact view-creation and iteration semantics should follow the implementation contract.

20 **B20:** Theta(sum from i=1 to t of `n_i`). Writing O(t times max `n_i`) is an upper bound but loses the tighter total-input description.

C. Debug-the-analysis solutions

1 Consecutive loops add rather than multiply. Three full scans are O(3n) = O(n).

2 Indexing one element is O(1); array length describes input storage, not lookup work.

3 Early return affects best case. If the match can be absent/last, worst-case time is O(n).

4 O(n) describes a growth class after constants and lower-order work, not exact machine instructions.

5 Use O(n + m) for two independently sized scans.

6 Analyze total movement. Two forward-only pointers can perform at most 2n advances, yielding O(n), assuming constant-time predicates.

7 Say expected O(1) under sound hashing and ordinary assumptions; do not promise it for every state/input.

8 Updating neighboring links may be O(1) after position is known, but reaching the middle by index is O(n), so total is O(n).

9 Only repeated `poll`/`remove` returns priority order. Iteration exposes heap-array order and takes O(n) to visit all entries.

10 Recursive calls consume stack frames. Space is proportional to maximum depth plus growing allocations.

11 Time counts all call-tree nodes; stack counts one active path. Naive Fibonacci has O(n) depth.

12 The constructor traverses and copies membership into new storage: O(n) time and O(n) logical slots.

13 Auxiliary space may exclude the required result, but the copied array still consumes O(n) result storage and should be reported separately.

14 For sufficiently large `n` under comparable constants, lower growth wins, but a particular O(n log n) implementation can be faster for relevant sizes or provide better guarantees/semantics.

15 Measurements compare executions in an environment; they do not prove behavior for arbitrarily growing `n`. Derivation establishes asymptotic complexity.

D. Coding-task solution guidance

D1. Linear search with inspection count

Return a small record such as `SearchResult(int index, int inspections)`. Increment before each equality check. Empty/absent input produces `(-1, n)`; a first-element match produces `(0, 1)`. See examples 02 and 03 in the companion.

D2. Rectangular cell visits

Use nested enhanced-for loops and increment one counter per cell. Time is O(rows times columns) only if rectangular; a general implementation naturally handles jagged rows in O(total cells).

D3. Halving steps

Repeatedly divide a positive value by two while it exceeds one. Define behavior for zero and negative input. Each step removes one binary magnitude level, so time is O(log n) and space O(1).

D4. Duplicate detection

The nested-loop version compares pairs: worst-case O(n squared), O(1) auxiliary space, no mutation. The HashSet version stops when `add` returns false: expected O(n) time, O(u) space, boxing for `int`, and dependency on sound equality/hash behavior.

D5. Frequency map

Use `counts.merge(value, 1, Integer::sum)` or `getOrDefault`. With n inputs and u distinct keys: expected O(n) time and O(u) result space. State boxing and hash assumptions.

D6. Two reverse contracts

The in-place two-pointer version is O(n) time/O(1) auxiliary space and mutates input. The safe version copies first, O(n) extra result storage, then reverses the copy. Neither is categorically superior; the API contract decides.

D7. Queue and stack

For a queue, use `offerLast` with `removeFirst`/`pollFirst`. For a stack, use `push`/`pop`/`peek`, or consistently use one end. Each end operation is amortized O(1); n operations are O(n) total.

D8. Largest k values

Maintain a min-heap of at most `k`. Offer until size `k`; for each later value larger than the head, replace the head. Time is O(n log k), auxiliary space O(k). If sorted output is required, draining adds O(k log k).

D9. Two-pointer proof

Identify a monotonic invariant. If `right` increments n times and `left` never retreats and increments at most n times, the total is at most 2n pointer moves. Include the predicate's cost.

D10. Jagged sum

Loop through each row and then its entries. If row i has `c_i` cells, time is Theta(sum `c_i`) and auxiliary space O(1), excluding the input.

D11. String construction

Use one StringBuilder and append each part. Define `C` as total characters appended; construction is O(C) in the ordinary amortized buffer-growth model and the result requires O(C) storage. Formatting/parsing inside the loop must be added separately.

D12. Deterministic duplicate API

Copy and sort the primitive array, then scan adjacent entries: O(n log n) time and O(n) copy/result storage, with no caller mutation. If mutation is permitted, sort in place. A tree-based set offers O(n log n) and O(n) storage. Discuss how the chosen primitive sort's documented guarantees and memory behavior affect the exact claim.

E. Interview follow-up model answers

1. Sorting avoids hash assumptions, can preserve ordered output, may use compact primitive arrays, and provides a predictable O(n log n) route; hashing uses extra structures but offers expected O(n).
2. Poor/adversarial distribution can increase collision work. Mutation of equality/hash fields can make an inserted key unreachable. Use sound immutable keys or choose an ordered/different representation.
3. Graph traversal is O(V + E) for adjacency-list representation because every vertex is processed and each stored edge is examined a constant number of times. An adjacency matrix changes edge-scan work to O(V squared).
4. Count aggregate pointer movement. If the inner pointer resets for every outer index, multiplication may be right; if it only advances across the entire method, O(n) may be right.
5. Returning all k matches prevents an early final answer and requires Omega(k) output work/storage. A typical scan becomes O(n) time and O(k) result space.
6. Amortization spreads rare expensive operations across a sequence. It does not cap individual request latency; one resize can still be O(n).
7. Always mention result/copy storage. Whether it is labeled auxiliary or output depends on the convention and contract, but capacity planning cannot pretend it is free.
8. DFS stack is O(h): O(log n) on a balanced tree and O(n) on a skewed tree. BFS is O(w), maximum width.
9. Name the probability source, such as sound hash distribution or randomized choices. Without a model, "average" is unsupported.
10. A benchmark can reveal constants, allocation, throughput, latency, and environment effects for tested sizes. It cannot establish asymptotic behavior over all larger inputs or repair a wrong cost model.
11. Mutation policy may require copies; memory limits may favor arrays/sorting; deterministic requirements may favor trees/sorting; lookup and ordering semantics select the implementation before micro-optimization.
12. First state the O(n squared) bottleneck, often repeated search. Introduce sorting or indexed membership, prove the new invariant/cost, report extra space/mutation/expected qualifiers, and validate representative edge cases.

F. Cumulative-assessment rubric

Assessment 1

A strong answer names n, derives O(1), O(n), O(log n), O(n), and O(n squared), and does not confuse source nesting with executed work.

Assessment 2

A strong answer connects ArrayList to indexing, HashSet to expected membership, TreeSet to O(log n) sorted navigation, ArrayDeque to amortized O(1) ends, and PriorityQueue to O(1) peek/O(log n) update. It mentions order/equality and avoids universal guarantees.

Assessment 3

A strong answer reports mutation and O(1) auxiliary space for in-place work, O(n) copy/result storage for defensive work, O(depth) recursive stack, and O(k) required output. It distinguishes shallow reference copying from deep copying.

Assessment 4

A strong answer identifies O(n) copy, sorting cost, repeated linear membership, potentially quadratic immutable concatenation, and O(k) output. It changes representations only when the method's contract supports them and states the resulting bounds.

Assessment 5

A strong explanation has a clear baseline, named input dimensions, bottleneck proof, improved invariant or data structure, qualified time and space, trade-off, and falsifying tests such as empty, smallest, worst-shape, duplicate-heavy, and maximum-size input.

Final-readiness scoring

Score each of the eight readiness bullets from the practice chapter:

- `2`: correct, concise, and independently explained;
- `1`: correct only with prompting or missing an important qualifier;
- `0`: incorrect or not attempted.

`14–16` means proceed. `10–13` means proceed only after revisiting the lowest-scoring chapter. Below `10` means repeat the foundations and practice lab. The goal is defensible reasoning, not memorized notation.

SDE-2 CORE CHAPTER

Chapter 9 - Twenty-Four Executable Complexity Examples

This dependency-free Java 21 companion validates behavior and operation counts for the volume's core growth shapes, space models, and collection choices. Compile with javac -Xlint:all -Werror ComplexityExamples.java and run with java ComplexityExamples. A successful run prints exactly PASS 24 complexity examples. Wall-clock timing is deliberately not used as proof of Big-O.

JAVA (continued 1/8)

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Set;
import java.util.TreeSet;

/**
 * Executable examples for Time and Space Complexity for Java Interviews.
 *
 * <p>These checks validate behavior and operation counts. They deliberately do
 * not use wall-clock timing as proof of asymptotic complexity.</p>
 */
public final class ComplexityExamples {
    private static int passed;

    private ComplexityExamples() {}

    public static void main(String[] args) {
        run("01 constant indexed access", ComplexityExamples::constantAccess);
        run("02 linear scan", ComplexityExamples::linearScan);
        run("03 early exit", ComplexityExamples::earlyExit);
        run("04 consecutive loops", ComplexityExamples::consecutiveLoops);
        run("05 independent nested loops", ComplexityExamples::independentNestedLoops);
        run("06 triangular loop", ComplexityExamples::triangularLoop);
        run("07 repeated halving", ComplexityExamples::repeatedHalving);
        run("08 logarithmic outer loop", ComplexityExamples::logarithmicOuterLoop);
        run("09 linearithmic work shape", ComplexityExamples::linearithmicShape);
        run("10 forward-only two pointers", ComplexityExamples::twoPointers);
```

JAVA (continued 2/8)

```
        run("11 geometric inner work", ComplexityExamples::geometricInnerWork);
        run("12 rectangular matrix", ComplexityExamples::rectangularMatrix);
        run("13 jagged matrix", ComplexityExamples::jaggedMatrix);
        run("14 StringBuilder growth", ComplexityExamples::stringBuilder);
        run("15 defensive array copy", ComplexityExamples::defensiveCopy);
        run("16 linear recursion depth", ComplexityExamples::linearRecursionDepth);
        run("17 logarithmic recursion depth", ComplexityExamples::logRecursionDepth);
        run("18 output-sensitive result", ComplexityExamples::outputSensitive);
        run("19 ArrayList indexed access", ComplexityExamples::arrayListAccess);
        run("20 HashSet membership", ComplexityExamples::hashSetMembership);
        run("21 HashMap frequency", ComplexityExamples::hashMapFrequency);
        run("22 TreeSet navigation", ComplexityExamples::treeSetNavigation);
        run("23 ArrayDeque queue", ComplexityExamples::arrayDequeQueue);
        run("24 PriorityQueue order", ComplexityExamples::priorityQueueOrder);

        System.out.println("PASS " + passed + " complexity examples");
    }

    private static void constantAccess() {
        int[] values = {5, 8, 13, 21};
        check(values[2] == 13, "indexed read");
    }

    private static void linearScan() {
        int[] values = {2, 4, 6, 8};
        ScanResult result = find(values, 99);
        check(result.index() == -1 && result.inspections() == values.length,
                "absent target inspects n values");
    }

    private static void earlyExit() {
        int[] values = {2, 4, 6, 8};
        ScanResult result = find(values, 2);
        check(result.index() == 0 && result.inspections() == 1,
```

JAVA (continued 3/8)

```
                "first target inspects once");
    }

    private static ScanResult find(int[] values, int target) {
        int inspections = 0;
        for (int index = 0; index < values.length; index++) {
            inspections++;
            if (values[index] == target) return new ScanResult(index, inspections);
        }
        return new ScanResult(-1, inspections);
    }

    private static void consecutiveLoops() {
        int n = 7;
        int operations = 0;
        for (int index = 0; index < n; index++) operations++;
        for (int index = 0; index < n; index++) operations++;
        check(operations == 2 * n, "n + n");
    }

    private static void independentNestedLoops() {
        int rows = 3;
        int columns = 5;
        int operations = 0;
        for (int row = 0; row < rows; row++) {
            for (int column = 0; column < columns; column++) operations++;
        }
        check(operations == rows * columns, "rows times columns");
    }

    private static void triangularLoop() {
        int n = 6;
        int operations = 0;
        for (int left = 0; left < n; left++) {
```

JAVA (continued 4/8)

```
            for (int right = left + 1; right < n; right++) operations++;
        }
        check(operations == n * (n - 1) / 2, "pair count");
    }

    private static void repeatedHalving() {
        int value = 32;
        int steps = 0;
        while (value > 1) {
            value /= 2;
            steps++;
        }
        check(steps == 5, "log2(32)");
    }

    private static void logarithmicOuterLoop() {
        int n = 16;
        int levels = 0;
        for (int size = 1; size < n; size *= 2) levels++;
        check(levels == 4, "1, 2, 4, 8");
    }

    private static void linearithmicShape() {
        int n = 16;
        int operations = 0;
        for (int size = 1; size < n; size *= 2) {
            for (int index = 0; index < n; index++) operations++;
        }
        check(operations == 64, "n times log2(n)");
    }

    private static void twoPointers() {
        int n = 9;
        int left = 0;
```

JAVA (continued 5/8)

```
        int movements = 0;
        for (int right = 0; right < n; right++) {
            movements++;
            while (left < right && right - left > 2) {
                left++;
                movements++;
            }
        }
        check(movements <= 2 * n, "each pointer advances at most n times");
    }

    private static void geometricInnerWork() {
        int n = 16;
        int operations = 0;
        for (int size = 1; size <= n; size *= 2) {
            for (int index = 0; index < size; index++) operations++;
        }
        check(operations == 31 && operations < 2 * n, "1 + 2 + ... + n");
    }

    private static void rectangularMatrix() {
        int[][] matrix = new int[2][5];
        int visits = 0;
        for (int[] row : matrix) {
            for (int ignored : row) visits++;
        }
        check(visits == 10, "rows times columns");
    }

    private static void jaggedMatrix() {
        int[][] matrix = {{1}, {2, 3, 4}, {}, {5, 6}};
        int visits = 0;
        for (int[] row : matrix) {
```

JAVA (continued 6/8)

```
            for (int ignored : row) visits++;
        }
        check(visits == 6, "sum of row lengths");
    }

    private static void stringBuilder() {
        List<String> parts = List.of("time", "-", "space");
        StringBuilder result = new StringBuilder();
        for (String part : parts) result.append(part);
        check(result.toString().equals("time-space"), "append characters");
    }

    private static void defensiveCopy() {
        int[] source = {3, 1, 2};
        int[] copy = Arrays.copyOf(source, source.length);
        copy[0] = 99;
        check(source[0] == 3 && copy[0] == 99, "independent arrays");
    }

    private static void linearRecursionDepth() {
        check(countDownDepth(7) == 8, "frames include base call");
    }

    private static int countDownDepth(int n) {
        if (n == 0) return 1;
        return 1 + countDownDepth(n - 1);
    }

    private static void logRecursionDepth() {
        check(halvingDepth(32) == 6, "32, 16, 8, 4, 2, 1");
    }

    private static int halvingDepth(int n) {
```

JAVA (continued 7/8)

```
        if (n <= 1) return 1;
        return 1 + halvingDepth(n / 2);
    }

    private static void outputSensitive() {
        int[] values = {4, 1, 4, 2, 4};
        List<Integer> positions = new ArrayList<>();
        for (int index = 0; index < values.length; index++) {
            if (values[index] == 4) positions.add(index);
        }
        check(positions.equals(List.of(0, 2, 4)), "output has k matches");
    }

    private static void arrayListAccess() {
        List<Integer> values = new ArrayList<>(List.of(10, 20, 30));
        check(values.get(1) == 20, "indexed access");
    }

    private static void hashSetMembership() {
        Set<Integer> seen = new HashSet<>();
        check(seen.add(7) && !seen.add(7), "add reports duplicate");
    }

    private static void hashMapFrequency() {
        Map<Integer, Integer> counts = new HashMap<>();
        for (int value : new int[] {2, 1, 2, 2}) {
            counts.merge(value, 1, Integer::sum);
        }
        check(counts.equals(Map.of(1, 1, 2, 3)), "frequency map");
    }

    private static void treeSetNavigation() {
        TreeSet<Integer> sorted = new TreeSet<>(List.of(8, 2, 5, 11));
```

JAVA (continued 8/8)

```
        check(sorted.ceiling(6) == 8 && sorted.floor(6) == 5, "ordered neighbors");
    }

    private static void arrayDequeQueue() {
        Deque<Integer> queue = new ArrayDeque<>();
        queue.offerLast(10);
        queue.offerLast(20);
        check(queue.removeFirst() == 10 && queue.removeFirst() == 20, "FIFO");
    }

    private static void priorityQueueOrder() {
        PriorityQueue<Integer> heap = new PriorityQueue<>();
        heap.addAll(List.of(9, 3, 7, 1));
        List<Integer> drained = new ArrayList<>();
        while (!heap.isEmpty()) drained.add(heap.remove());
        check(drained.equals(List.of(1, 3, 7, 9)), "poll order");
    }

    private static void run(String name, Runnable example) {
        try {
            example.run();
            passed++;
        } catch (RuntimeException | AssertionError error) {
            throw new AssertionError("FAILED " + name, error);
        }
    }

    private static void check(boolean condition, String message) {
        if (!condition) throw new AssertionError(message);
    }

    private record ScanResult(int index, int inspections) {}
}
```

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: count simple, sequential, logarithmic, and nested work
- Interview Core: analyze two pointers, recursion, strings, copying, and collections
- SDE-2 Follow-up: defend amortized, expected, output-sensitive, and ownership trade-offs
- Challenge: complete the cumulative and final readiness assessments without notes

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: Study Step 01 - Java Foundations for Problem Solving

Series index

Next book: Study Step 03A - Number Systems and Math Foundations for DSA Interviews

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the study steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. The same public study codes appear on the website and every PDF cover. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
01	Java Foundations for Problem Solving
02	Time and Space Complexity for Java Interviews
03A	Number Systems and Math Foundations for DSA Interviews
03B	Number Systems Interview Patterns and Rapid Revision
04	Bit Manipulation in Java
05	Loop Mastery, Patterns, and Index Calculations
06	Arrays and Array Problem-Solving Patterns
07	Strings and String Problem-Solving Patterns
08	Hashing: Maps, Sets, Frequency, and Prefix State
09	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18A	Advanced Java A: JVM and Execution
18B	Advanced Java B: Language, OOP, and Modern Java
18C	Advanced Java C: Collections, Streams, and I/O
18D	Advanced Java D: Concurrency and the Memory Model
18E	Advanced Java E: Performance, Diagnostics, and GC Incidents
18F	Advanced Java F: Design, Backend, Testing, and Security
18G	Advanced Java G: Question Bank, Study Plan, and Reference
18H	Advanced Java H: Spring Boot and REST APIs
18I	Advanced Java I: Persistence, SQL, JPA, and Caching
18J	Advanced Java J: Distributed Systems and System Design

Study Step 03 uses linked foundations (03A) and workbook (03B) books. Study Step 18 uses ten focused books (18A-18J), including a three-book backend specialist track. These suffixes keep every file focused while preserving one unambiguous route.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Study Step 02 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.